Desafio: Análise de logs

Para realização do desafio de forma eficiente e não precisar analisar cada linha manualmente, criei um script em Java no IntelliJ IDEA para fazer a leitura automatizada, extração dos campos e classificação dos eventos. Basicamente ele retorna a analise e a classficiação dos logs em poucas linhas utilizando a amostra dos logs e a correlação que retorna o IP destino e a quantidade de conexões em curto periodo que esse ip recebeu.

2. Metodologia e Desenvolvimento do Script

Para organizar a análise do desafio, dividi a lógica do código em três etapas principais, simulando os níveis de atuação de um SOC (n1, n2 e n3):

Nível 1 (n1) - Triagem e Leitura: O código lê as linhas de texto brutas do arquivo de log e utiliza Expressões Regulares (regex) para extrair os campos principais de cada evento, como horário (time), host de origem, ação executada, IP de origem, IP de destino, quantidade de bytes e usuário. Eventos comuns sem nenhuma suspeita são classificaddos como tráfego normal.

Nível 2 (n2) - Análise de Ameaças: No código, criei regras específicas de verificação para identificar suspeitas de ataque

- Acessos diretos com o usuário root no servidor srv-linux-01, que indicam o uso de credenciais privilegiadas fora do padrão.
- Tráfego de rede no proxy (Proxy-Edge-01) direcionado ao IP 10.0.0.1 com um tamanho fixo de 2450 bytes, o que é um sinal característico de comunicação automatizada de vírus ou malware (Beaconing / Command & Control).

Nível 3 (n3) - Correlação de Dados: No final da execução, o programa agrupa os eventos pelo IP de destino e faz uma contagem para verificar se o mesmo IP recebeu múltiplos acessos em um curto periodo, gerando um alerta de correlação quando isso acontece.

3. Código Criado para o Desafio

Abaixo está o código Java completo desenvolvido e utilizado para o processamento e classificação dos logs:

```
import java.util.*;
import java.util.regex.*;
import java.util.stream.Collectors;

public class Main {

   // modelagem do log
   public record LogEvent(String timestamp, String host, String
action, String srcIp, String destIp, int bytes, String user) {}
```

```
    // extracao de campos do log
    private static LogEvent parseLine(String line) {
        String timestamp = extractValue(line, "time=([^\\s]+)");
        String host = extractValue(line, "host=([^\\s]+)");
        String action = extractValue(line, "action=([^\\s]+)");
        String srcIp = extractValue(line, "src_ip=([^\\s]+)");
        String destIp = extractValue(line, "dest_ip=([^\\s]+)");
        String bytesStr = extractValue(line, "bytes=([^\\s]+)");
        String user = extractValue(line, "user=([^\\s]+)");

        int bytes = bytesStr != null ? Integer.parseInt(bytesStr) : 0;

        return new LogEvent(timestamp, host, action, srcIp, destIp,
bytes, user);
    }

    private static String extractValue(String line, String regex) {
        Matcher matcher = Pattern.compile(regex).matcher(line);
        return matcher.find() ? matcher.group(1) : null;
    }

    // regras de classificacao SOC n1 e n2
    private static String classificarAmeaca(LogEvent log) {
        if ("Proxy-Edge-01".equals(log.host()) && log.bytes() == 2450
&& "10.0.0.1".equals(log.destIp())) {
            return "CRÍTICO - N2: Possível Malware Beaconing / C2
(Tráfego padronizado)";
        }

        if ("srv-linux-01".equals(log.host()) &&
"root".equals(log.user()) && "accepted".equals(log.action())) {
            return "ALTO - N2: Login de Root aceito fora do padrão";
        }

        return "INFO - N1: Tráfego Normal";
    }

    public static void main(String[] args) {
```

```
        // amostra de logs para demonstracao mais limpa e clara no
terminal
        List<String> linhasDeLogRaw = Arrays.asList(
                "time=10:00:01 host=Proxy-Edge-01 action=allow
src_ip=10.0.0.24 dest_ip=10.0.0.1 bytes=2450",
                "time=10:00:02 host=srv-linux-01 action=accepted
src_ip=10.0.0.48 dest_ip=10.0.1.5 bytes=120 user=root",
                "time=10:00:03 host=SRV-WEB-01 action=success
src_ip=10.0.0.15 dest_ip=10.0.1.10 bytes=500 user=c_santos",
                "time=10:00:04 host=Proxy-Edge-01 action=allow
src_ip=10.0.0.83 dest_ip=10.0.0.1 bytes=2450"
        );
        //inicialiazacao de analise dos logs do ficheiro
dataset_lab.log
        System.out.println("=== INICIANDO ANÁLISE DE LOGS ===\n");

        List<LogEvent> eventosProcessados = linhasDeLogRaw.stream()
                .map(Main::parseLine)
                .collect(Collectors.toList());

        // processsamento n1/n2
        for (LogEvent evento : eventosProcessados) {
            String classificacao = classificarAmeaca(evento);
            System.out.printf("[%s] Host: %s | Origem: %s -> Destino:
%s | Classificação: %s%n",
                    evento.timestamp(), evento.host(), evento.srcIp(),
evento.destIp(), classificacao);
        }

        System.out.println("\n=== CORRELAÇÃO DE DADOS (N3) ===");

        // modulo de correlacao n3
        Map<String, Long> acessosPorDestino =
eventosProcessados.stream()
                .collect(Collectors.groupingBy(LogEvent::destIp,
Collectors.counting()));

        acessosPorDestino.forEach((ip, contagem) -> {
            if (contagem > 1) {
```

```
                System.out.println("ALERTA DE CORRELAÇÃO: O IP destino
" + ip + " recebeu " + contagem + " conexões em curto período!");
            }
        });
    }
}
```

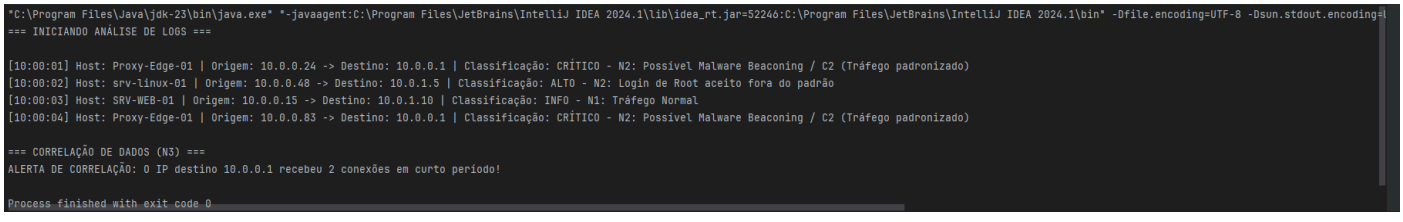
4. Resultados e Análise dos Logs

Ao executar o script no IntelliJ retornou a seguinte saída no terminal:
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.1\lib\idea_rt.jar=52246:C:\Program Files\JetBrains\IntelliJ IDEA 2024.1\bin" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\Aluno\IdeaProjects\analiseLogs\out\production\analiseLogs Main
=== INICIANDO ANÁLISE DE LOGS ===

[10:00:01] Host: Proxy-Edge-01 | Origem: 10.0.0.24 -> Destino: 10.0.0.1 | Classificação: CRÍTICO - N2: Possível Malware Beaconing / C2 (Tráfego padronizado)
[10:00:02] Host: srv-linux-01 | Origem: 10.0.0.48 -> Destino: 10.0.1.5 | Classificação: ALTO - N2: Login de Root aceito fora do padrão
[10:00:03] Host: SRV-WEB-01 | Origem: 10.0.0.15 -> Destino: 10.0.1.10 | Classificação: INFO - N1: Tráfego Normal
[10:00:04] Host: Proxy-Edge-01 | Origem: 10.0.0.83 -> Destino: 10.0.0.1 | Classificação: CRÍTICO - N2: Possível Malware Beaconing / C2 (Tráfego padronizado)

=== CORRELAÇÃO DE DADOS (N3) ===
ALERTA DE CORRELAÇÃO: O IP destino 10.0.0.1 recebeu 2 conexões em curto período!

Process finished with exit code 0



Com base nos resultados gerados pelo script, foi possível identificar duas ameaças principais:
1. Um acesso bem sucedido com a conta root no servidor Linux vindo de uma máquina da rede

interna. Isso é crítico pois logins de admins direto devem ser restritos ou investigados.
2. Duas máquinas diferentes (10.0.0.24 e 10.0.0.83) enviando pacotes com exatamente o mesmo tamanho (2450 bytes) para o IP 10.0.0.1. A regra n3 pegou esse padrão repetido de conexões no mesmo destino, confirmando o alerta de correlação para tráfego suspeito de malware.

O desenvolvimento do script ajudou a resolver o desafio de forma automatizada e rápida. Conseguimos separar o tráfego comum dos eventos suspeitos (n1 e n2) e, no final, correlacionar os acessos no N3 para comprovar o comportamento anômalo da rede. O código rodou perfeitamente e atendeu ao que foi pedido no desafio.